

DESARROLLO DE APLICACIONES WEB AVANZADO

LABORATORIO N° 9 INTRODUCCIÓN NEXT.JS



Alumnos		Nota
Grupo		
Fecha de Entrega		
Docente	Omar Castañeda Rodríguez	

OBJETIVOS:

Comprender el funcionamiento básico de Next.js.

Diferenciar entre CSR (Client Side Rendering) y SSR (Server Side Rendering).

SEGURIDAD:



Advertencia:

En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red; así como la ingestión de alimentos o bebidas.

FUNDAMENTO TEÓRICO:

- Revisar el texto guía que está en el campus Virtual.

NORMAS EMPLEADAS:

- No aplica

RECURSOS:

- En este laboratorio cada alumno trabajará con un equipo con Windows 11.

METODOLOGÍA PARA EL DESARROLLO DE LA TAREA:

- El desarrollo del laboratorio es individual

PROCEDIMIENTO:

Requisitos Previos

- Tener instalado Node.js (<https://nodejs.org>)
- Tener instalado npm o yarn
- Editor de código como Visual Studio Code

Ejercicio 1: Hola Mundo en Next.js

1. Abre Visual Studio Code.
2. Crea una nueva carpeta: nextjs-lab
3. Abre la terminal en esa carpeta:

```
npx create-next-app@latest
```

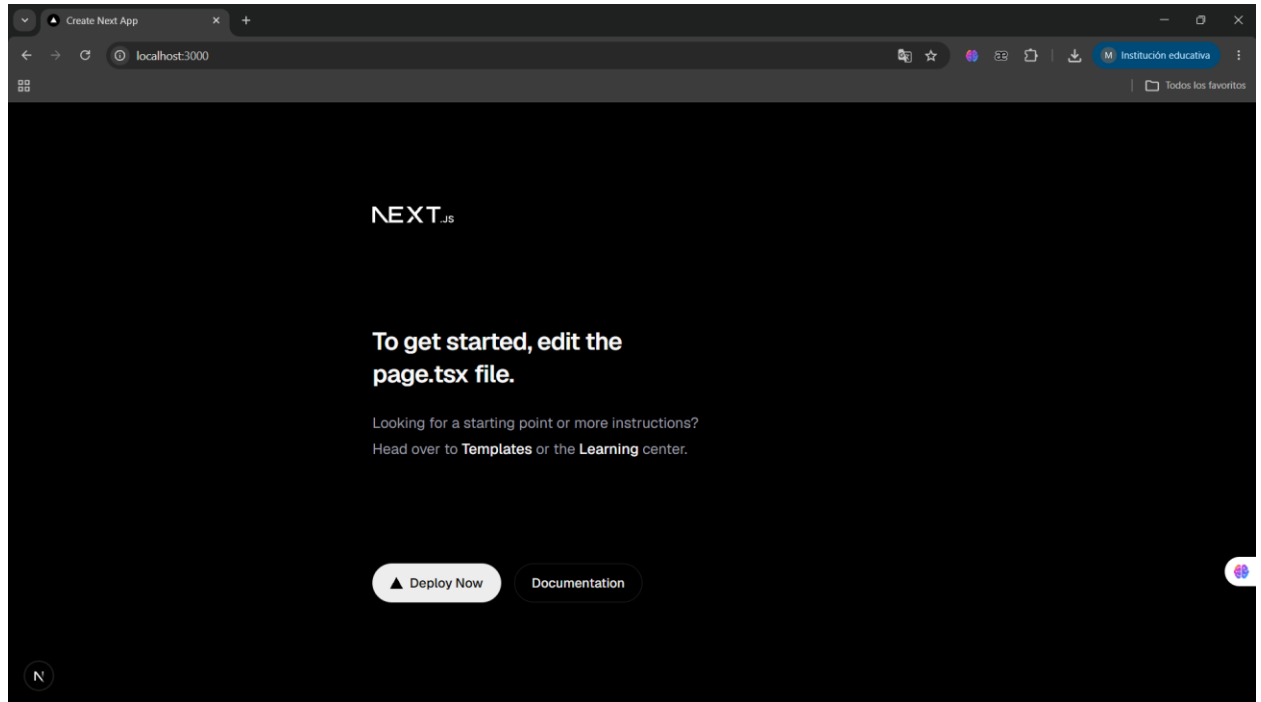
4. Acepta los valores de la siguiente forma:

```
✓ What is your project named? ... hello-world
✓ Would you like to use TypeScript? ... No / Yes
✓ Would you like to use ESLint? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like your code inside a `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to use Turbopack for `next dev`? ... No / Yes
✓ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
```

5. Ingresa al proyecto creado e inicia el servidor:

```
cd hello-world
npm run dev
```

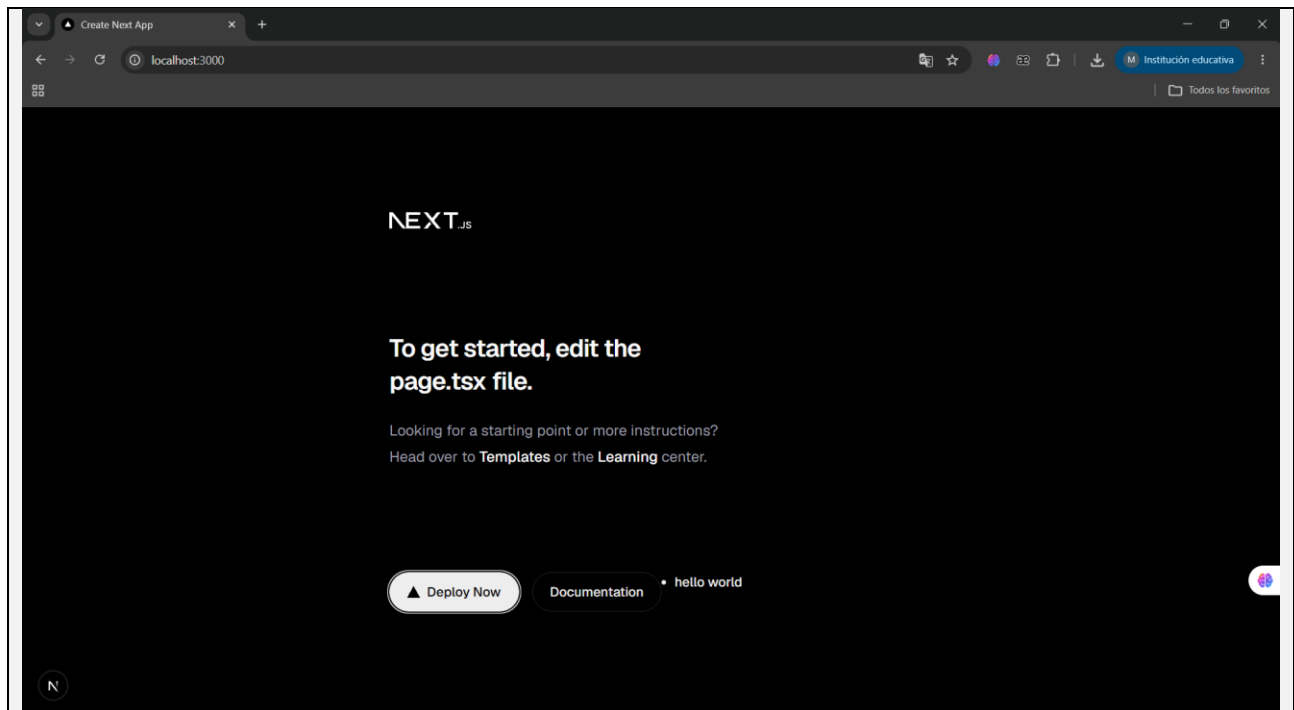
6. Luego nos ubicamos en nuestro navegador e ingresamos a **localhost:3000**



7. Modifica el archivo `src/app/page.tsx` y coloca el siguiente código:

```
1
2 <ol className="list-inside list-decimal text-sm/6 text-center sm:text-left font-[family-name:var(--font-geist-mono)]">
3   <li className="mb-2 tracking-[-.01em]">
4     Get started by editing(" ")
5     <code className="bg-black/[.05] dark:bg-white/[.06] px-1 py-0.5 rounded font-[family-name:var(--font-geist-mono)] font-semibold">
6       src/app/page.tsx
7     </code>
8   </li>
9
10  <li className="tracking-[-.01em]">
11    Save and see your changes instantly.
12  </li>
13  <li className="tracking-[-.01em]">
14    Hello world!.
15  </li>
16</ol>
```

Captura del resultado



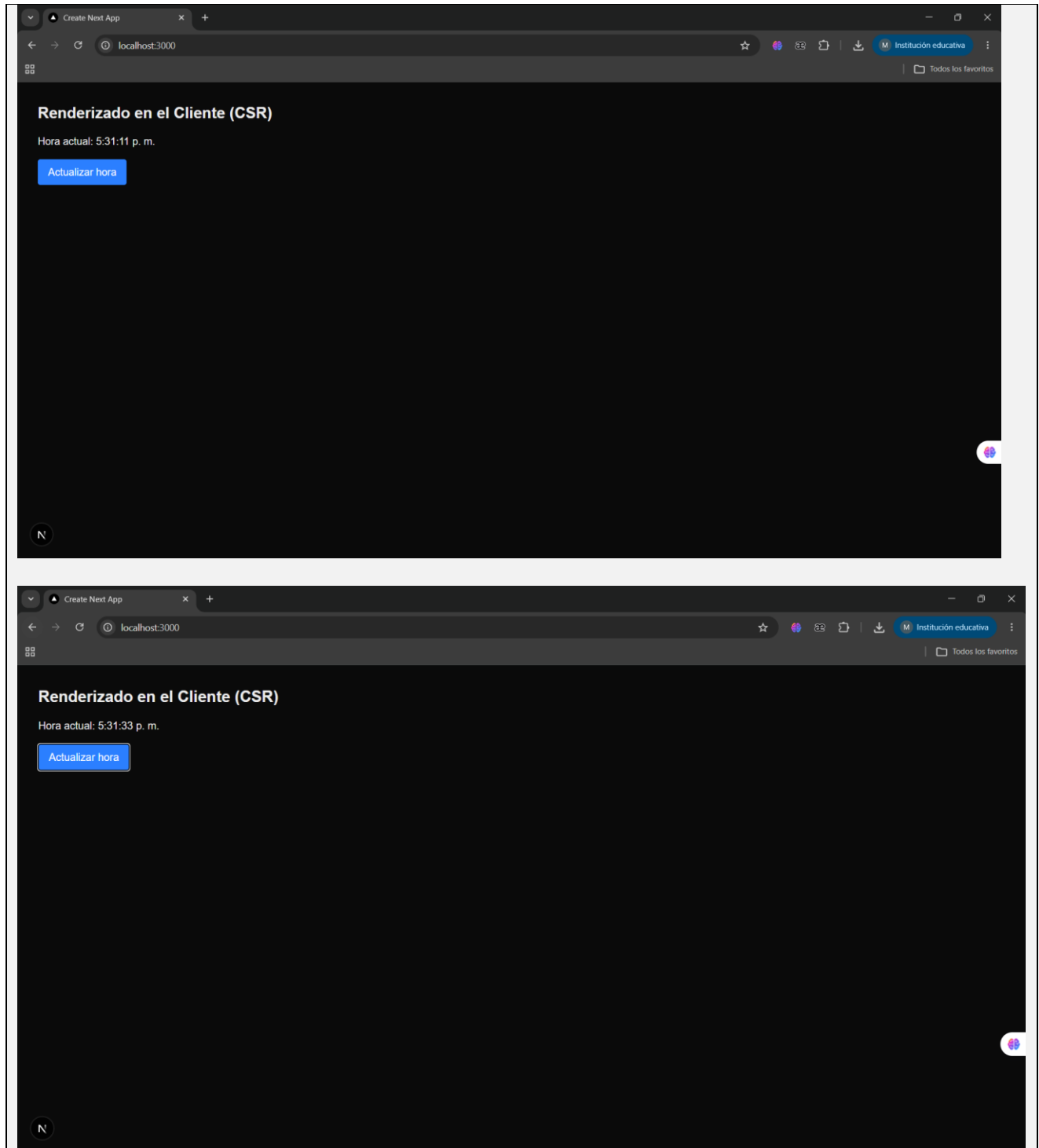
Ejercicio 2: CSR (Client-Side Rendering)

Crear un componente que muestre la hora actual y se actualice al hacer clic.

1. Crea un nuevo proyecto con next
2. Acepta los valores por defecto
3. Modifica el archivo src/app/page.tsx y coloca el siguiente código:

```
1  'use client';
2
3  import { useState } from 'react';
4
5  export default function CSRPage() {
6    const [hora, setHora] = useState(new Date().toLocaleTimeString());
7
8    const actualizarHora = () => {
9      setHora(new Date().toLocaleTimeString());
10   };
11
12   return (
13     <main className="p-8">
14       <h1 className="text-2xl font-bold mb-4">Renderizado en el Cliente (CSR)</h1>
15       <p>Hora actual: {hora}</p>
16       <button
17         onClick={actualizarHora}
18         className="mt-4 px-4 py-2 bg-blue-500 text-white rounded"
19       >
20         Actualizar hora
21       </button>
22     </main>
23   );
24 }
```

Captura



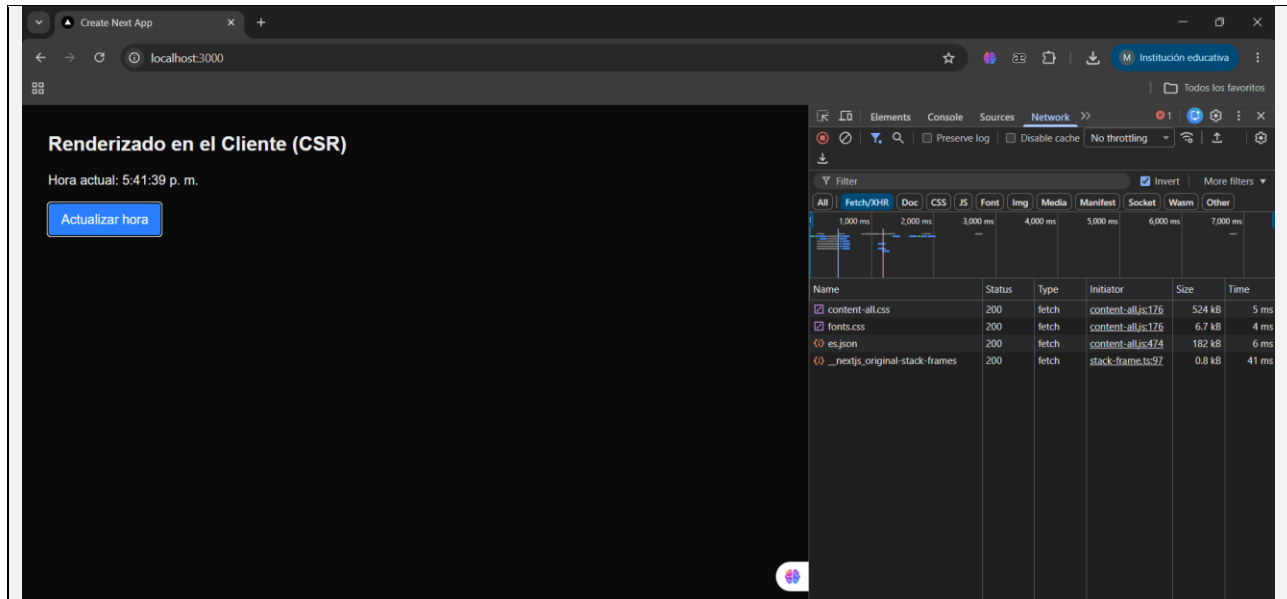
Ejercicio 3: SSR (Server-Side Rendering)

Mostrar la hora actual renderizada desde el servidor.

1. Crea un nuevo proyecto con next
2. Acepta los valores por defecto
3. Modifica el archivo `src/app/page.tsx` y coloca el siguiente código:

```
1  'use client';
2
3  import { useState } from 'react';
4
5  export default function CSRPage() {
6    const [hora, setHora] = useState(new Date().toLocaleTimeString());
7
8    const actualizarHora = () => {
9      setHora(new Date().toLocaleTimeString());
10   };
11
12   return (
13     <main className="p-8">
14       <h1 className="text-2xl font-bold mb-4">Renderizado en el Cliente (CSR)</h1>
15       <p>Hora actual: {hora}</p>
16       <button
17         onClick={actualizarHora}
18         className="mt-4 px-4 py-2 bg-blue-500 text-white rounded"
19       >
20         Actualizar hora
21       </button>
22     </main>
23   );
24 }
```

Captura



TAREA

Lista de Productos con Filtro por Categoría (CSR)

Objetivo:

- Mostrar una lista de productos.
- Filtrar productos por categoría.
- Usar useState y useEffect.

Instrucciones:

- Crea una página en /app/tarea/page.tsx
- Muestra una lista de productos usando CSR.

- Agrega un botón que permita filtrar los productos por categoría.
- Usa al menos un useState y un useEffect.

Lista de Productos (CSR)

Todas

Tecnología

Ropa

Alimentos

Mostrando 9 producto(s) — categoría: Todas

Laptop HP

Tecnología

S/ 2500.00

Mouse Logitech

Tecnología

S/ 80.00

Teclado Mecánico

Tecnología

S/ 250.00

Camiseta Polo

Ropa

S/ 60.00

Pantalón Jean

Ropa

S/ 120.00

Zapatillas Nike

Ropa

S/ 320.00

Pan integral

Alimentos

S/ 8.00

Leche entera 1L

Alimentos

S/ 5.00

Manzanas (1kg)

Alimentos

S/ 7.00

N

← → ↻ http://localhost:3000/tarea

Lista de Productos (CSR)

Todas **Tecnología** Ropa Alimentos

Mostrando 3 producto(s) — categoría: Tecnología

Laptop HP Tecnología S/ 2500.00	Mouse Logitech Tecnología S/ 80.00	Teclado Mecánico Tecnología S/ 250.00
---	--	---

N

The screenshot shows a web browser window with the address bar displaying 'http://localhost:3000/tarea'. The page title is 'Lista de Productos (CSR)'. Below the title, there are four filter buttons: 'Todas', 'Tecnología', 'Ropa' (which is highlighted in blue), and 'Alimentos'. Below the filters, a message states 'Mostrando 3 producto(s) — categoría: Ropa'. The main content area displays three product cards, each with a title, category, and price:

Producto	Categoría	Precio
Camiseta Polo	Ropa	S/ 60.00
Pantalón Jean	Ropa	S/ 120.00
Zapatillas Nike	Ropa	S/ 320.00

In the bottom left corner of the application, there is a circular button with a right-pointing arrow.

Colocar capturas del código y capturas de ejecución

Grabar explicando el código y ejecución de este en máximo 4 minutos

OBSERVACIONES: *(Las observaciones son las notas aclaratorias, objeciones y problemas que se pudo presentar en el desarrollo del laboratorio)*

1. El renderizado inicial en CSR puede mostrar contenido vacío hasta que el JavaScript se ejecuta en el navegador.
2. Mezclar SSR con `new Date()` en estado inicial provoca errores de hidratación por diferencias entre servidor y cliente.
3. Generar las categorías dinámicamente con `new Set()` evita tener que actualizarlas manualmente al agregar productos.
4. Tailwind CSS agiliza el diseño visual sin necesidad de archivos CSS adicionales.
5. La ruta `/tarea` se crea automáticamente al colocar `page.tsx` dentro de `src/app/tarea/`, gracias al file-based routing de Next.js.

CONCLUSIONES: *(Las conclusiones son una opinión sobre tu trabajo, explicar cómo resolviste las dudas o problemas presentados en el laboratorio. Además de aportar una opinión crítica de lo realizado)*

1. CSR permite construir interfaces interactivas que reaccionan a las acciones del usuario sin recargar la página.
2. `useState` y `useEffect` son la base para manejar estado dinámico y efectos secundarios en componentes cliente.
3. La directiva `'use client'` es obligatoria en Next.js cuando se necesitan hooks de React en el App Router.
4. Filtrar datos en el cliente es eficiente para listas pequeñas, ya que no requiere peticiones extra al servidor.
5. Estructurar los productos como un array tipado con TypeScript facilita el mantenimiento y reduce errores.